Machine Intelligence 1 Tutorial

02 Function Fitting

Fachgebiet Neuronale Informationsverarbeitung (NI)

WiSe 25/26

The plan for today

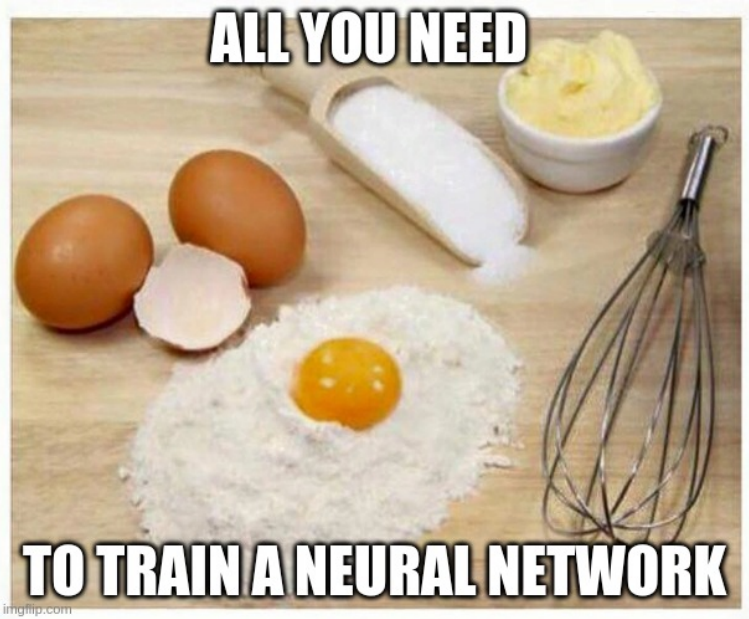
Ingredients for function fitting

- Cost functions

Gradient-based learning

- Finding the minimum of the training error analytically
- Gradient descent
- Gradient calculation: Connectionist neuron

Ingredients for function fitting



Tuning the weights of a model to perform some task.

Ingredients for function fitting

1. A cost function: $$e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) \stackrel{!}{=} \min_{\underline{\mathbf{w}}}$$

Ingredients for function fitting

1. A cost function: $e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) \stackrel{!}{=} \min_{\underline{\mathbf{w}}}$
2. A performance measure, a criterion for *model selection*.

$$E^G_{[\underline{\mathbf{w}}]} := \langle e \rangle_{y_T, \underline{\mathbf{x}}; \underline{\mathbf{w}}} = \iint d\underline{\mathbf{x}}\, dy_T\; P(y_T, \underline{\mathbf{x}})\, e(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right))$$

Because $P(y_T, \underline{\mathbf{x}})$ is not known,
empirical average $E^T_{[\underline{\mathbf{w}}]}$ using the available training data:

$$\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)} \right) \right\}, \alpha = 1, \ldots, p$$

. batch training error: $E^T_{[\underline{\mathbf{w}}]} = \frac{1}{p} \sum_{\alpha=1}^{p} \underbrace{e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}} \right) \right)}_{e^{(\alpha)}}$

Ingredients for function fitting

1. A cost function: $e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) \stackrel{!}{=} \min_{\underline{\mathbf{w}}}$
2. A performance measure, a criterion for *model selection*.

$$E^G_{[\underline{\mathbf{w}}]} := \langle\, e \,\rangle_{y_T, \underline{\mathbf{x}}; \underline{\mathbf{w}}} = \iint d\underline{\mathbf{x}}\, dy_T\; P(y_T, \underline{\mathbf{x}})\, e(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right))$$

$$E^T_{[\underline{\mathbf{w}}]} = \frac{1}{p} \sum_{\alpha=1}^{p} \underbrace{e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}_{e^{(\alpha)}}$$

3. A model with tunable parameters $\underline{\mathbf{w}}$: connectionist neuron, MLP,...

Ingredients for function fitting

1. A cost function: $$e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) \stackrel{!}{=} \min_{\underline{\mathbf{w}}}$$
2. A performance measure, a criterion for *model selection*.

$$E^G_{[\underline{\mathbf{w}}]} := \langle e \rangle_{y_T, \underline{\mathbf{x}}; \underline{\mathbf{w}}} = \iint d\underline{\mathbf{x}}\, dy_T\; P(y_T, \underline{\mathbf{x}})\, e(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right))$$

$$E^T_{[\underline{\mathbf{w}}]} = \frac{1}{p} \sum_{\alpha=1}^{p} \underbrace{e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}_{e^{(\alpha)}}$$

3. A model with tunable parameters $\underline{\mathbf{w}}$: connectionist neuron, MLP,...
4. A learning algorithm

Cost functions

Choosing a cost function accounts for

1. the type of problem (i.e. regression vs. classification),
2. how the model is penalized for different types of mistakes it can make such as: small errors are tolerable, large errors are penalized less, etc.

Cost functions for Regression

$$\underbrace{\underline{\mathbf{x}} \in \mathbb{R}^N}_{\text{feature vector}} \longrightarrow \underbrace{y \in \mathbb{R}}_{\substack{\text{scalar}\\ \text{attribute}}}$$

$y_T(\underline{\mathbf{x}})$: true value of attribute

$y(\underline{\mathbf{x}}; \underline{\mathbf{w}})$: predicted value of attribute (e.g. by MLP)

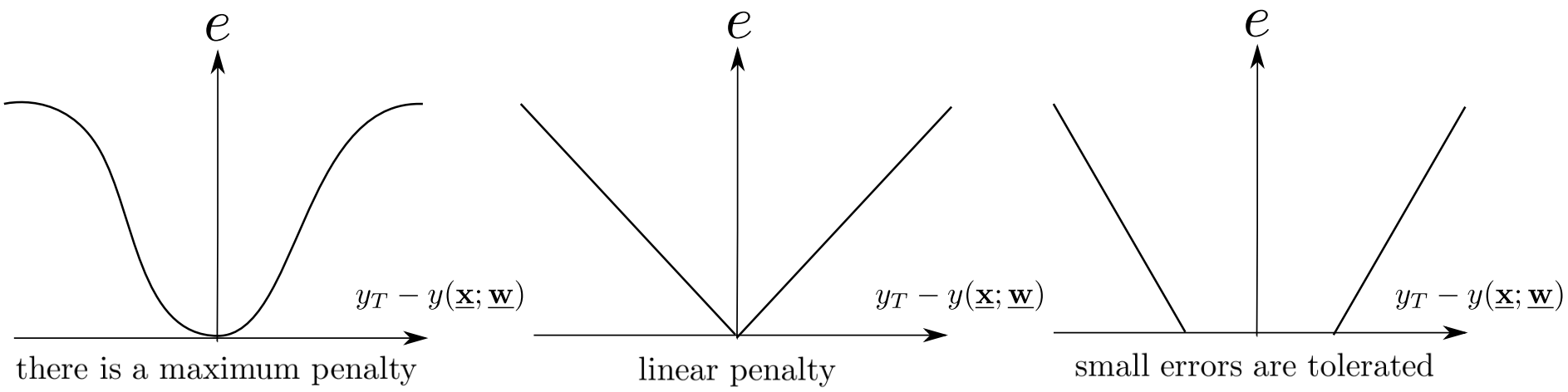
Cost functions for Regression

$$\underbrace{\underline{\mathbf{x}} \in \mathbb{R}^N}_{\text{feature vector}} \longrightarrow \underbrace{y \in \mathbb{R}}_{\text{scalar attribute}}$$

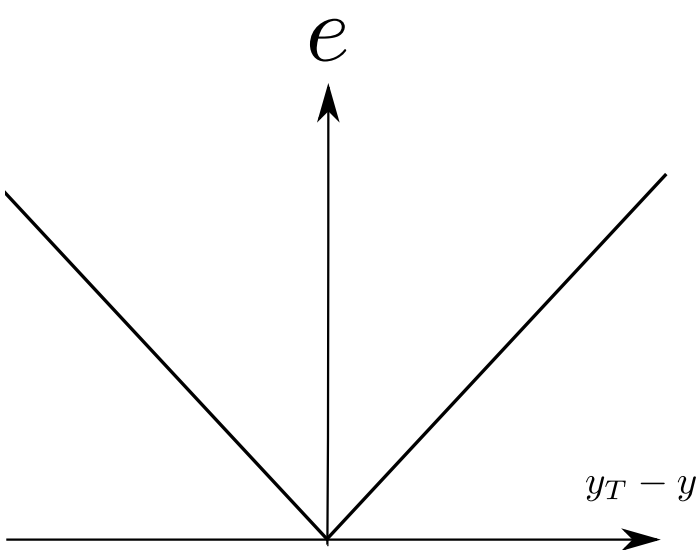
$y_T(\underline{\mathbf{x}})$: true value of attribute

$y(\underline{\mathbf{x}}; \underline{\mathbf{w}})$: predicted value of attribute (e.g. by MLP)

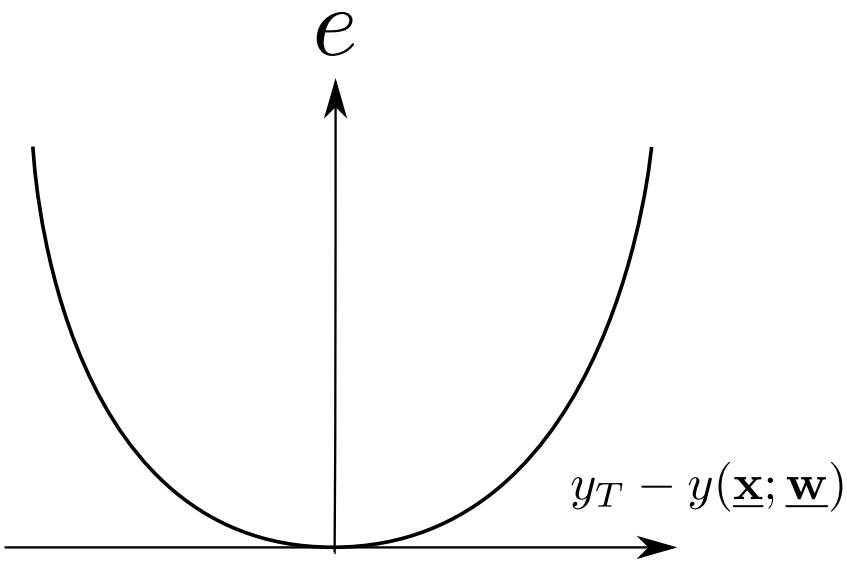
individual cost $e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right)$



Quadratic vs. linear error



(a) linear error

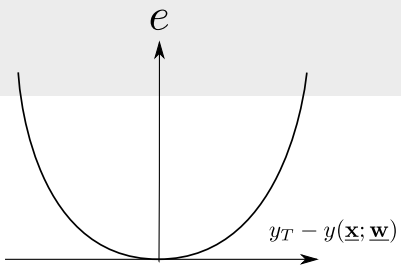


(b) qudratic error

$$e_{\text{quadratic}}\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) := \frac{1}{2}\Big(y_T(\underline{\mathbf{x}}) - y(\underline{\mathbf{x}}; \underline{\mathbf{w}})\Big)^2$$

$$e_{\text{linear}}\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) := \Big|y_T(\underline{\mathbf{x}}) - y(\underline{\mathbf{x}}; \underline{\mathbf{w}})\Big|$$

Quadratic vs. linear error



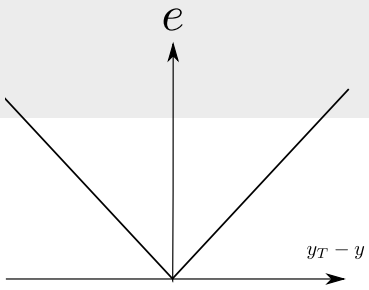


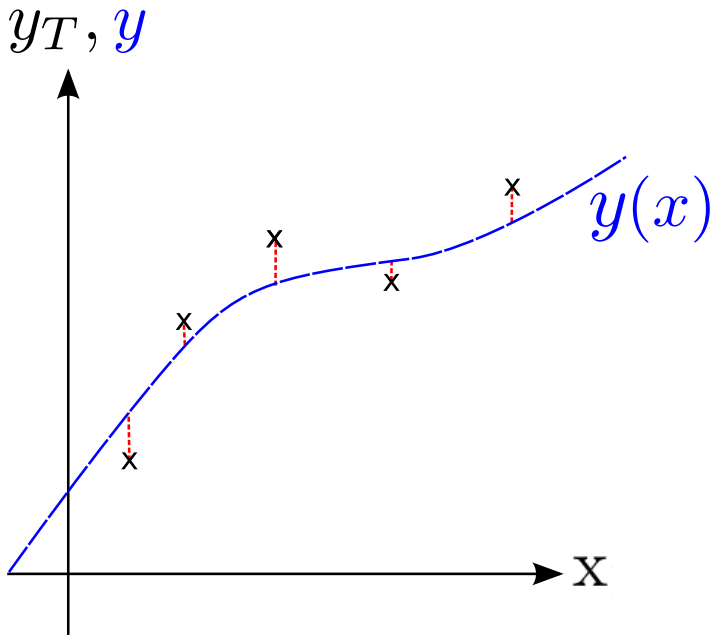
Table: Differences between quadratic and linear cost

quadratic	linear
larger error $\rightsquigarrow$ larger penalty (sensitive to outliers)	constant increase in error (more stable, robust to outliers)
converges faster but slow convergence for small errors	constant convergence rate
differentiable everywhere	not differentiable at zero (not a huge problem)
max. likelihood of Gaussian variable	

Relation between quadratic error and Gaussian labels

Assume labels y_T are conditionally Gaussian:

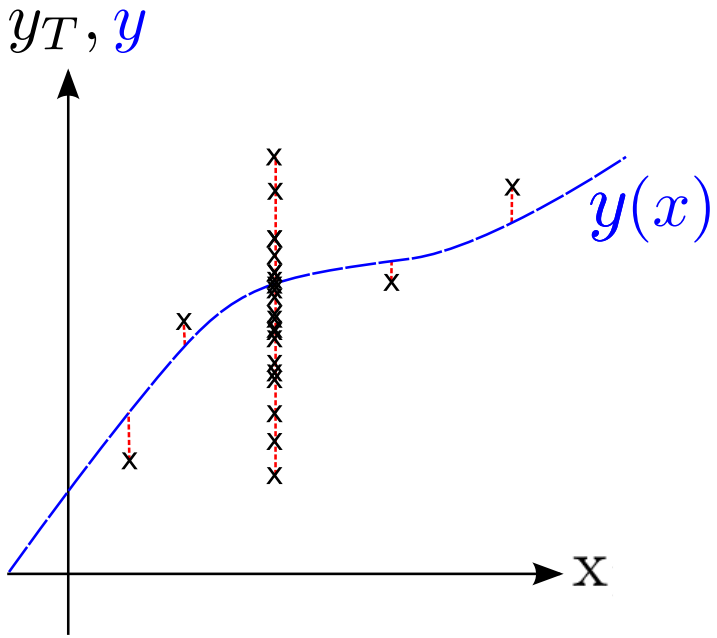
$$P(y_T|\underline{\mathbf{x}}) = \mathcal{N}(y_T|\underbrace{y(\underline{\mathbf{x}};\underline{\mathbf{w}})}_{=\mu}, \sigma^2)$$



Relation between quadratic error and Gaussian labels

Assume labels y_T are conditionally Gaussian:

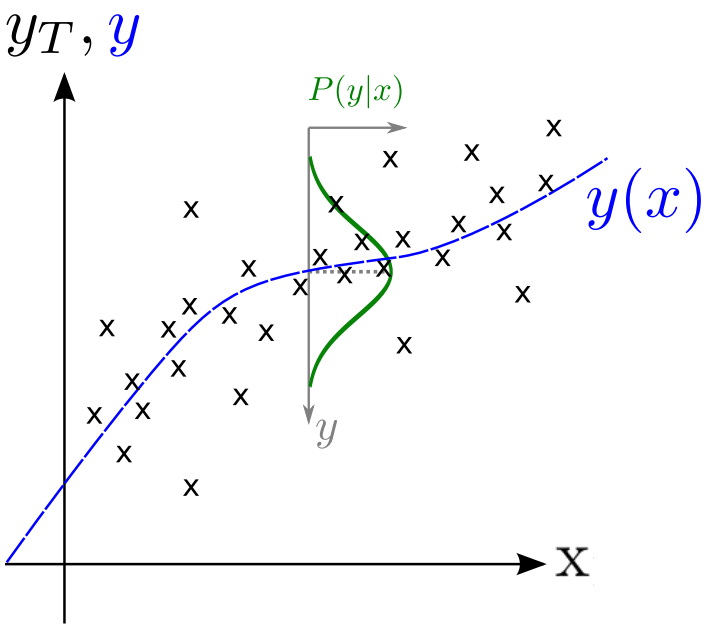
$$P(y_T|\underline{\mathbf{x}}) = \mathcal{N}(y_T|\underbrace{y(\underline{\mathbf{x}};\underline{\mathbf{w}})}_{=\mu}, \sigma^2)$$



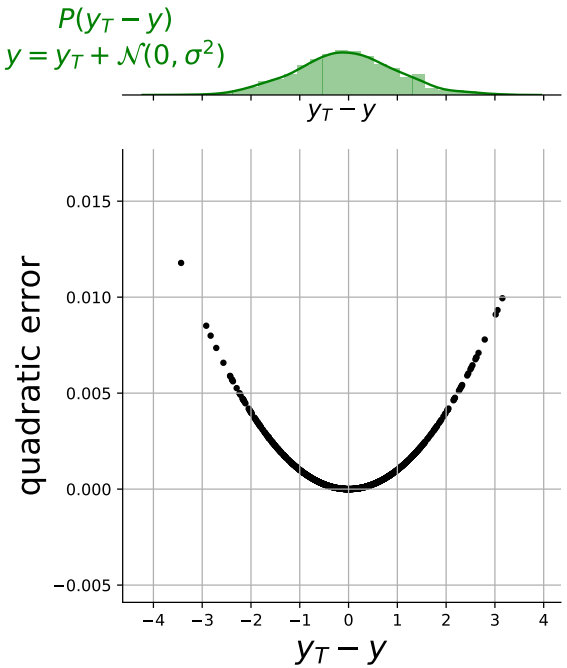
Relation between quadratic error and Gaussian labels

Assume labels y_T are conditionally Gaussian:

$$P(y_T|\underline{\mathbf{x}}) = \mathcal{N}(y_T|\underbrace{y(\underline{\mathbf{x}};\underline{\mathbf{w}})}_{=\mu}, \sigma^2)$$



(a) guassian labels



(b) aggregate density

Relation between quadratic error and Gaussian labels

Assume labels y_T are conditionally Gaussian:

$$P(y_T|\underline{\mathbf{x}}) = \mathcal{N}(y_T|\underbrace{y(\underline{\mathbf{x}};\underline{\mathbf{w}})}_{=\mu},\sigma^2)$$

$$\underline{\mathbf{w}}^* \in \underset{\underline{\mathbf{w}}}{\operatorname{argmin}}\, E^T_{[\underline{\mathbf{w}}]} \iff \underset{\underline{\mathbf{w}}}{\operatorname{argmax}} \underbrace{\prod_{\alpha=1}^{p} \mathcal{N}(y_T|y(\underline{\mathbf{x}};\underline{\mathbf{w}}),\sigma^2)}_{\text{likelihood function } \mathcal{L}(\underline{\mathbf{w}})}$$

$$= \underset{\underline{\mathbf{w}}}{\operatorname{argmin}} \underbrace{[-\ln \mathcal{L}(\underline{\mathbf{w}})]}_{\text{neg. log likelihood}}$$

Cost function for binary classification
Deriving the cross entropy cost function for binary classification

Data:

$$\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)} \right) \right\}_{\alpha=1}^{p}$$

with

$$\underline{\mathbf{x}}^{(\alpha)} \in \mathbb{R}^N \quad , \quad y_T^{(\alpha)} \in \{0, 1\}.$$

generated by

$$\left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)} \right) \overset{i.i.d.}{\sim} P_{\text{data}}(\underline{\mathbf{x}}, y)$$

Cost function for binary classification
Deriving the cross entropy cost function for binary classification

Data:

$$\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)} \right) \right\}_{\alpha=1}^{p}$$

with

$$\underline{\mathbf{x}}^{(\alpha)} \in \mathbb{R}^N \quad , \quad y_T^{(\alpha)} \in \{0, 1\}.$$

generated by

$$\left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)} \right) \overset{i.i.d.}{\sim} P_{\text{data}}(\underline{\mathbf{x}}, y)$$

Model:
MLP, scalar output

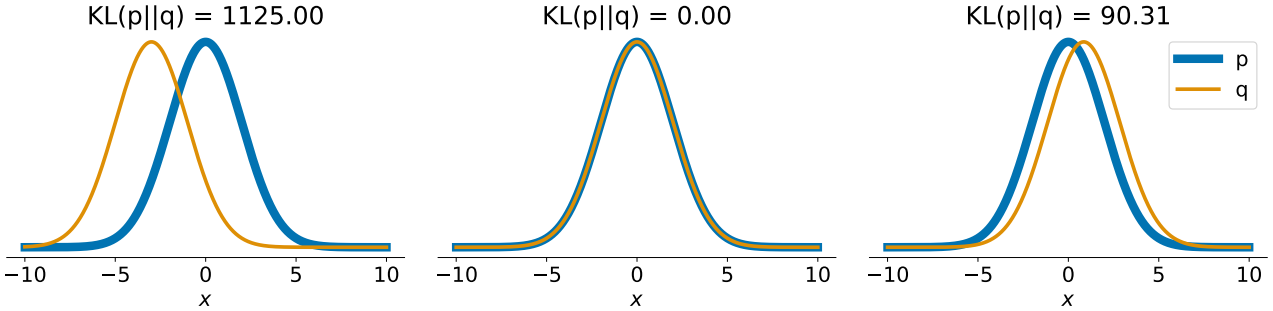
$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) =: P_{\text{model}}(y = 1 | \underline{\mathbf{x}}; \underline{\mathbf{w}}) \text{ and } P_{\text{model}}(y = 0 | \underline{\mathbf{x}}; \underline{\mathbf{w}}) = 1 - y(\underline{\mathbf{x}}; \underline{\mathbf{w}})$$

for the "positive" and "negative"/"other" class respectively.

Cost function derivation based on the Kullback-Leibler divergence

$\mathrm{D_{KL}}(P||Q)$ is a measure of how much a probability distribution P differs from another distribution Q.

$$\mathrm{D_{KL}}\left(P(z)|| \, Q(z) \right) := \int dz \, P(z) \ln \left(\frac{P(z)}{Q(z)} \right)$$



Cost function derivation based on the KL-divergence

$\mathrm{D_{KL}}(P||Q)$ is a measure of how much a probability distribution P differs from another distribution Q.

$$\mathrm{D_{KL}}\left(P(z)|| \, Q(z) \right) := \int dz \, P(z) \ln \left(\frac{P(z)}{Q(z)} \right)$$

Properties of the KL-divergence:

- $\mathrm{D_{KL}} = 0$ iff. both distribution are *identical*,
- otherwise, $\mathrm{D_{KL}} > 0$.

! Does not qualify as a metric because it is not symmetric. For different distributions P and Q:

$$\mathrm{D_{KL}}(P||Q) \neq \mathrm{D_{KL}}(Q||P)$$

From KL-divergence to binary cross entropy loss

$$\mathrm{D_{KL}}\left(P_{\text{data}}(\underline{\mathbf{x}}, y) \,||\, P_{\text{model}}(\underline{\mathbf{x}}, y) \right) = \int\int d\underline{\mathbf{x}}\; dy P_{\text{data}}(\underline{\mathbf{x}}, y) \ln\left(\frac{P_{\text{data}}(\underline{\mathbf{x}}, y)}{P_{\text{model}}(\underline{\mathbf{x}}, y)} \right)$$

$$= \int_{\mathbb{R}^N} d\underline{\mathbf{x}} \sum_{y\in\{0,1\}} P_{\text{data}}(\underline{\mathbf{x}}, y) \ln\left(\frac{P_{\text{data}}(\underline{\mathbf{x}}, y)}{P_{\text{model}}(\underline{\mathbf{x}}, y)} \right)$$

$$= \int_{\mathbb{R}^N} d\underline{\mathbf{x}} \sum_{y\in\{0,1\}} \underbrace{P_{\text{data}}(\underline{\mathbf{x}}) P_{\text{data}}(y|\underline{\mathbf{x}})}_{\substack{=P_{\text{data}}(\underline{\mathbf{x}},y) \\ \text{(Chain rule)}}} \ln\left(\frac{P_{\text{data}}(\underline{\mathbf{x}}) P_{\text{data}}(y|\underline{\mathbf{x}})}{\underbrace{P_{\cancel{\text{model}}}(\underline{\mathbf{x}})}_{\substack{\text{discriminative model} \\ \text{not a generative model} \\ \text{replace with } P_{\text{data}}(\underline{\mathbf{x}})}} \; P_{\text{model}}(y|\underline{\mathbf{x}})} \right)$$

From KL-divergence to binary cross entropy loss

$$
\begin{aligned}
&\mathrm{D}_{\mathrm{KL}}\left(P_{\text{data}}(\underline{\mathbf{x}}, y) \,||\, P_{\text{model}}(\underline{\mathbf{x}}, y) \right) \\
&= \int_{\mathbb{R}^N} d\underline{\mathbf{x}} \sum_{y\in\{0,1\}} P_{\text{data}}(\underline{\mathbf{x}}, y) \ln\left(\frac{P_{\text{data}}(\underline{\mathbf{x}}, y)}{P_{\text{model}}(\underline{\mathbf{x}}, y)} \right) \\
&= \int_{\mathbb{R}^N} d\underline{\mathbf{x}} \sum_{y\in\{0,1\}} \underbrace{P_{\text{data}}(\underline{\mathbf{x}}) P_{\text{data}}(y|\underline{\mathbf{x}})}_{\substack{=P_{\text{data}}(\underline{\mathbf{x}},y) \\ \text{(Chain rule)}}} \ln\left(\frac{P_{\text{data}}(\underline{\mathbf{x}}) P_{\text{data}}(y|\underline{\mathbf{x}})}{\underbrace{\cancel{P_{\text{model}}(\underline{\mathbf{x}})}}_{\substack{\text{discriminative model} \\ \text{not a generative model} \\ \text{replace with } P_{\text{data}}(\underline{\mathbf{x}})}} \; P_{\text{model}}(y|\underline{\mathbf{x}})} \right) \\
&= \int_{\mathbb{R}^N} d\underline{\mathbf{x}} \sum_{y\in\{0,1\}} \underbrace{P_{\text{data}}(\underline{\mathbf{x}})}_{\text{indep. of } y} P_{\text{data}}(y|\underline{\mathbf{x}}) \ln\left(\frac{P_{\text{data}}(\underline{\mathbf{x}}) P_{\text{data}}(y|\underline{\mathbf{x}})}{P_{\text{data}}(\underline{\mathbf{x}}) P_{\text{model}}(y|\underline{\mathbf{x}})} \right)
\end{aligned}
$$

From KL-divergence to binary cross entropy loss

$$= \int_{\mathbb{R}^N} d\underline{\mathbf{x}} \sum_{y\in\{0,1\}} \underbrace{P_{\text{data}}(\underline{\mathbf{x}})}_{\text{indep. of } y} P_{\text{data}}(y|\underline{\mathbf{x}}) \ln\left(\frac{P_{\text{data}}(\underline{\mathbf{x}})P_{\text{data}}(y|\underline{\mathbf{x}})}{P_{\text{data}}(\underline{\mathbf{x}})P_{\text{model}}(y|\underline{\mathbf{x}})}\right)$$

$$= \underbrace{\int_{\mathbb{R}^N} d\underline{\mathbf{x}}\, P_{\text{data}}(\underline{\mathbf{x}}) \sum_{y\in\{0,1\}} P_{\text{data}}(y|\underline{\mathbf{x}}) \ln[P_{\text{data}}(y|\underline{\mathbf{x}})]}_{\text{indep. of model parameters } \underline{\mathbf{w}}}$$

$$- \int_{\mathbb{R}^N} d\underline{\mathbf{x}}\, P_{\text{data}}(\underline{\mathbf{x}}) \underbrace{\sum_{y\in\{0,1\}} P_{\text{data}}(y|\underline{\mathbf{x}}) \ln[P_{\text{model}}(y|\underline{\mathbf{x}})]}_{\textbf{cross entropy}\text{ between data \& model distributions given } \underline{\mathbf{x}}}$$

From KL-divergence to binary cross entropy loss

$$= \int_{\mathbb{R}^N} d\underline{\mathbf{x}} \sum_{y\in\{0,1\}} \underbrace{P_{\text{data}}(\underline{\mathbf{x}})}_{\text{indep. of } y} P_{\text{data}}(y|\underline{\mathbf{x}}) \ln\left(\frac{P_{\text{data}}(\underline{\mathbf{x}})P_{\text{data}}(y|\underline{\mathbf{x}})}{P_{\text{data}}(\underline{\mathbf{x}})P_{\text{model}}(y|\underline{\mathbf{x}})}\right)$$

$$= \underbrace{\int_{\mathbb{R}^N} d\underline{\mathbf{x}}\, P_{\text{data}}(\underline{\mathbf{x}}) \sum_{y\in\{0,1\}} P_{\text{data}}(y|\underline{\mathbf{x}}) \ln[P_{\text{data}}(y|\underline{\mathbf{x}})]}_{\text{indep. of model parameters } \underline{\mathbf{w}}}$$

$$- \int_{\mathbb{R}^N} d\underline{\mathbf{x}}\, P_{\text{data}}(\underline{\mathbf{x}}) \underbrace{\sum_{y\in\{0,1\}} P_{\text{data}}(y|\underline{\mathbf{x}}) \ln[P_{\text{model}}(y|\underline{\mathbf{x}})]}_{\substack{\textbf{cross entropy} \text{ between} \\ \text{data \& model distributions given } \underline{\mathbf{x}}}}$$

Apply ERM on the second term with **cross entropy**.

ERM

$$
\begin{aligned}
E^T_{[\underline{\mathbf{w}}]} &= \frac{1}{p}\sum_{\alpha=1}^{p} -\Bigg(\sum_{y\in\{0,1\}} P_{\text{data}}(y|\underline{\mathbf{x}}^{(\alpha)}) \ln[P_{\text{model}}(y|\underline{\mathbf{x}}^{(\alpha)})] \Bigg) \\
&= \frac{1}{p}\sum_{\alpha=1}^{p} -\Bigg(P_{\text{data}}(y=1|\underline{\mathbf{x}}^{(\alpha)}) \ln[P_{\text{model}}(y=1|\underline{\mathbf{x}}^{(\alpha)})] \\
&\qquad + P_{\text{data}}(y=0|\underline{\mathbf{x}}^{(\alpha)}) \ln[P_{\text{model}}(y=0|\underline{\mathbf{x}}^{(\alpha)})] \Bigg)
\end{aligned}
$$

ERM

$$
\begin{aligned}
E^T_{[\underline{\mathbf{w}}]} &= \frac{1}{p}\sum_{\alpha=1}^{p} -\Bigg(\sum_{y\in\{0,1\}} P_{\mathsf{data}}(y|\underline{\mathbf{x}}^{(\alpha)}) \ln[P_{\mathsf{model}}(y|\underline{\mathbf{x}}^{(\alpha)})] \Bigg) \\
&= \frac{1}{p}\sum_{\alpha=1}^{p} -\Bigg(P_{\mathsf{data}}(y=1|\underline{\mathbf{x}}^{(\alpha)}) \ln[P_{\mathsf{model}}(y=1|\underline{\mathbf{x}}^{(\alpha)})] \\
&\qquad\qquad + P_{\mathsf{data}}(y=0|\underline{\mathbf{x}}^{(\alpha)}) \ln[P_{\mathsf{model}}(y=0|\underline{\mathbf{x}}^{(\alpha)})] \Bigg) \\
&= \frac{1}{p}\sum_{\alpha=1}^{p} -\Bigg(y_T^{(\alpha)} \cdot \ln[y(\underline{\mathbf{x}}^{(\alpha)};\underline{\mathbf{w}})] + (1-y_T^{(\alpha)}) \cdot \ln[1-y(\underline{\mathbf{x}}^{(\alpha)};\underline{\mathbf{w}})] \Bigg)
\end{aligned}
$$

ERM

$$
\begin{aligned}
E^T_{[\underline{\mathbf{w}}]} &= \frac{1}{p}\sum_{\alpha=1}^{p} -\Bigg(\sum_{y\in\{0,1\}} P_{\text{data}}(y|\underline{\mathbf{x}}^{(\alpha)}) \ln[P_{\text{model}}(y|\underline{\mathbf{x}}^{(\alpha)})] \Bigg) \\
&= \frac{1}{p}\sum_{\alpha=1}^{p} -\Bigg(P_{\text{data}}(y=1|\underline{\mathbf{x}}^{(\alpha)}) \ln[P_{\text{model}}(y=1|\underline{\mathbf{x}}^{(\alpha)})] \\
&\qquad\qquad + P_{\text{data}}(y=0|\underline{\mathbf{x}}^{(\alpha)}) \ln[P_{\text{model}}(y=0|\underline{\mathbf{x}}^{(\alpha)})] \Bigg) \\
&= \frac{1}{p}\sum_{\alpha=1}^{p} -\Bigg(y_T^{(\alpha)} \cdot \ln[y(\underline{\mathbf{x}}^{(\alpha)};\underline{\mathbf{w}})] + (1-y_T^{(\alpha)}) \cdot \ln[1-y(\underline{\mathbf{x}}^{(\alpha)};\underline{\mathbf{w}})] \Bigg) \\
&= \frac{1}{p}\sum_{\alpha=1}^{p} \Bigg(-y_T^{(\alpha)} \cdot \ln[y(\underline{\mathbf{x}}^{(\alpha)};\underline{\mathbf{w}})] - (1-y_T^{(\alpha)}) \cdot \ln[1-y(\underline{\mathbf{x}}^{(\alpha)};\underline{\mathbf{w}})] \Bigg)
\end{aligned}
$$

task	data $\left\{\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)}\right\}_{\alpha=1}^{p}$	output layer	cost function $e := e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right)$
scalar regression	$y_T \in \mathbb{R}$	linear $y = \underline{\mathbf{w}}^{LL-1\top} \underline{\mathbf{s}}^{L-1}$	quadratic error $e = \frac{1}{2}\left(y_T - y\right)^2$

task	data $\left\{\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)}\right\}_{\alpha=1}^{p}$	output layer	cost function $e := e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right)$
scalar regression	$y_T \in \mathbb{R}$	linear $y = \underline{\mathbf{w}}^{LL-1\top} \underline{\mathbf{s}}^{L-1}$	quadratic error $e = \frac{1}{2}\left(y_T - y\right)^2$
multivariate linear regression	$\underline{\mathbf{y}}_T \in \mathbb{R}^M$	linear $y_k = \underline{\mathbf{w}}_k^{LL-1\top} \underline{\mathbf{s}}^{L-1}$	squared Euclidean distance $e = \frac{1}{2}\lVert \underline{\mathbf{y}}_T - \underline{\mathbf{y}} \rVert_2^2$

task	data $\{\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)}\}_{\alpha=1}^{p}$	output layer	cost function $e := e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right)$
scalar regression	$y_T \in \mathbb{R}$	linear $y = \underline{\mathbf{w}}^{LL-1\top} \underline{\mathbf{s}}^{L-1}$	quadratic error $e = \frac{1}{2}\left(y_T - y\right)^2$
multivariate linear regression	$\underline{\mathbf{y}}_T \in \mathbb{R}^M$	linear $y_k = \underline{\mathbf{w}}_k^{LL-1\top} \underline{\mathbf{s}}^{L-1}$	squared Euclidean distance $e = \frac{1}{2}\lVert \underline{\mathbf{y}}_T - \underline{\mathbf{y}} \rVert_2^2$
binary classification	$y_T \in \{0,1\}, \{-1,1\}$	logistic sigmoid $y = \frac{1}{1+exp(-h_1^L)}$ or $y = \tanh(h_1^L)$	cross entropy $e = -y_T \ln[y] - (1-y_T)\ln[1-y]$

task	data $\{\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)}\}_{\alpha=1}^{p}$	output layer	cost function $e := e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right)$
scalar regression	$y_T \in \mathbb{R}$	linear $y = \underline{\mathbf{w}}^{LL-1\top} \underline{\mathbf{s}}^{L-1}$	quadratic error $e = \frac{1}{2}\left(y_T - y\right)^2$
multivariate linear regression	$\underline{\mathbf{y}}_T \in \mathbb{R}^M$	linear $y_k = \underline{\mathbf{w}}_k^{LL-1\top} \underline{\mathbf{s}}^{L-1}$	squared Euclidean distance $e = \frac{1}{2}\lVert \underline{\mathbf{y}}_T - \underline{\mathbf{y}} \rVert_2^2$
binary classification	$y_T \in \{0,1\}, \{-1,1\}$	logistic sigmoid $y = \frac{1}{1+exp(-h_1^L)}$ or $y = \tanh(h_1^L)$	cross entropy $e = -y_T \ln[y] - (1-y_T)\ln[1-y]$
multi-class classification with M classes	$\underline{\mathbf{y}}_T \in \{0,1\}^M$ 1-out-of-M encoding/ 1-hot-encoding e.g. $\underline{\mathbf{y}}_T = (0,0,1,0)^\top$ $\widehat{=}$ class 3 out of 4	softmax $y_k = \frac{exp(h_k^L)}{\sum_{k=1}^M exp(h_k^L)}$ $\Rightarrow \sum_{k=1}^M y_k = 1$	cross entropy (multi-class case) $e = -\sum_{k=1}^M y_{Tk} \cdot \ln[y_k\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)]$

Where are we?

Ingredients for function fitting

Gradient-based learning

Iterative, step-by-step minimization of the training error.

Minimizing the training error

Training error E^T for the training set $\left\{(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)})\right\}$:

$$E^T_{[\underline{\mathbf{w}}]} = \frac{1}{p}\sum_{\alpha=1}^{p} e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)$$

$$E^T_{[\underline{\mathbf{w}}]} \stackrel{!}{=} \min_{\underline{\mathbf{w}}} \quad \Rightarrow \quad \underline{\mathbf{w}}^* = \operatorname*{argmin}_{\underline{\mathbf{w}}} E^T_{[\underline{\mathbf{w}}]}$$

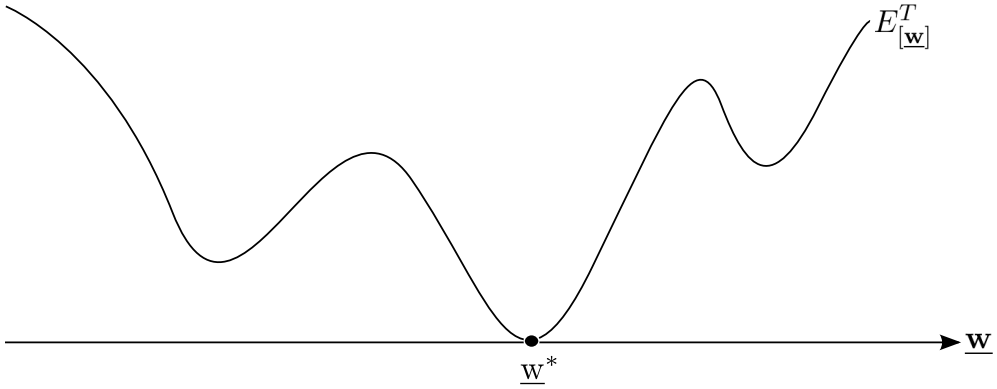


Figure: Training error with minimum at $\underline{\mathbf{w}}^*$

Minimizing the training error

Training error E^T for the training set $\left\{(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)})\right\}$:

$$E^T_{[\underline{\mathbf{w}}]} = \frac{1}{p}\sum_{\alpha=1}^{p} e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)$$

$$E^T_{[\underline{\mathbf{w}}]} \stackrel{!}{=} \min_{\underline{\mathbf{w}}} \quad \Rightarrow \quad \underline{\mathbf{w}}^* = \operatorname*{argmin}_{\underline{\mathbf{w}}} E^T_{[\underline{\mathbf{w}}]}$$

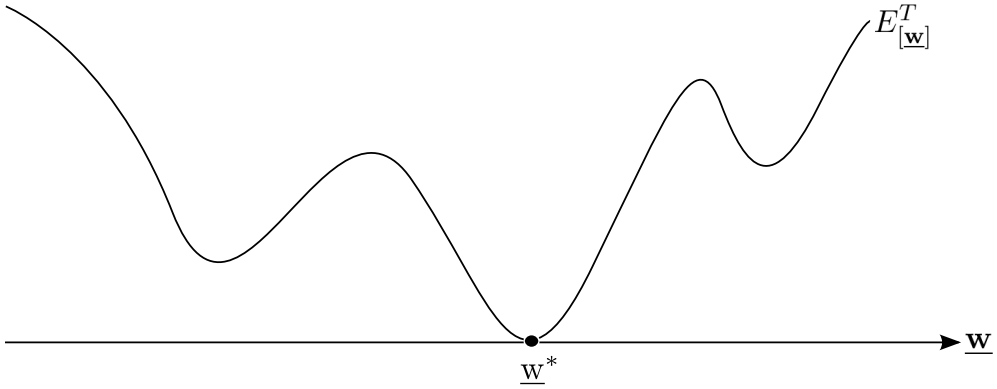


Figure: Training error with minimum at $\underline{\mathbf{w}}^*$

Q: *What is the strategy for finding $\underline{\mathbf{w}}^*$ analytically?*

Finding the minimum of the training error analytically

Example with a simple connectionist neuron model with parameters

$$\underline{\mathbf{w}} = (w_0, w_1, \ldots, w_N)^\top$$

The strategy for finding the minimum of a function $E^T_{[\underline{\mathbf{w}}]}$ analytically:

1. Compute the gradient w.r.t. $\underline{\mathbf{w}}$ by taking the first partial derivatives w.r.t to each component in the vector $\underline{\mathbf{w}}$

$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial \underline{\mathbf{w}}} = \left(\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_0}, \frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_1}, \ldots, \frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_N} \right)^\top$$

The gradient $\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial \underline{\mathbf{w}}}$ has the same dimensionality as $\underline{\mathbf{w}}$.

Finding the minimum of the training error analytically

Example with a simple connectionist neuron model with parameters

$$\underline{\mathbf{w}} = (w_0, w_1, \ldots, w_N)^\top$$

The strategy for finding the minimum of a function $E^T_{[\underline{\mathbf{w}}]}$ analytically:

1. Compute the gradient w.r.t. $\underline{\mathbf{w}}$ by taking the first partial derivatives w.r.t to each component in the vector $\underline{\mathbf{w}}$

$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial \underline{\mathbf{w}}} = \left(\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_0}, \frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_1}, \ldots, \frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_N} \right)^\top$$

2. Set the gradient to zero: $\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial \underline{\mathbf{w}}} \stackrel{!}{=} \underline{\mathbf{0}}$

Finding the minimum of the training error analytically

Example with a simple connectionist neuron model with parameters

$$\underline{\mathbf{w}} = (w_0, w_1, \ldots, w_N)^\top$$

The strategy for finding the minimum of a function $E^T_{[\underline{\mathbf{w}}]}$ analytically:

1. Compute the gradient w.r.t. $\underline{\mathbf{w}}$ by taking the first partial derivatives w.r.t to each component in the vector $\underline{\mathbf{w}}$

$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial \underline{\mathbf{w}}} = \left(\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_0}, \frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_1}, \ldots, \frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_N} \right)^\top$$

2. Set the gradient to zero: $\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial \underline{\mathbf{w}}} \stackrel{!}{=} \underline{\mathbf{0}}$
3. Solve for $\underline{\mathbf{w}}$ to find extrema.

Finding the minimum of the training error analytically

Example with a simple connectionist neuron model with parameters

$$\underline{\mathbf{w}} = (w_0, w_1, \ldots, w_N)^\top$$

The strategy for finding the minimum of a function $E^T_{[\underline{\mathbf{w}}]}$ analytically:

1. Compute the gradient w.r.t. $\underline{\mathbf{w}}$ by taking the first partial derivatives w.r.t to each component in the vector $\underline{\mathbf{w}}$

$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial \underline{\mathbf{w}}} = \left(\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_0}, \frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_1}, \ldots, \frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_N} \right)^\top$$

2. Set the gradient to zero: $\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial \underline{\mathbf{w}}} \stackrel{!}{=} \underline{\mathbf{0}}$
3. Solve for $\underline{\mathbf{w}}$ to find extrema.
4. Select solution corresponding to global minimum.

Finding the minimum of the training error analytically

Example with a simple connectionist neuron model with parameters

$$\underline{\mathbf{w}} = (w_0, w_1, \ldots, w_N)^\top$$

The strategy for finding the minimum of a function $E^T_{[\underline{\mathbf{w}}]}$ analytically:

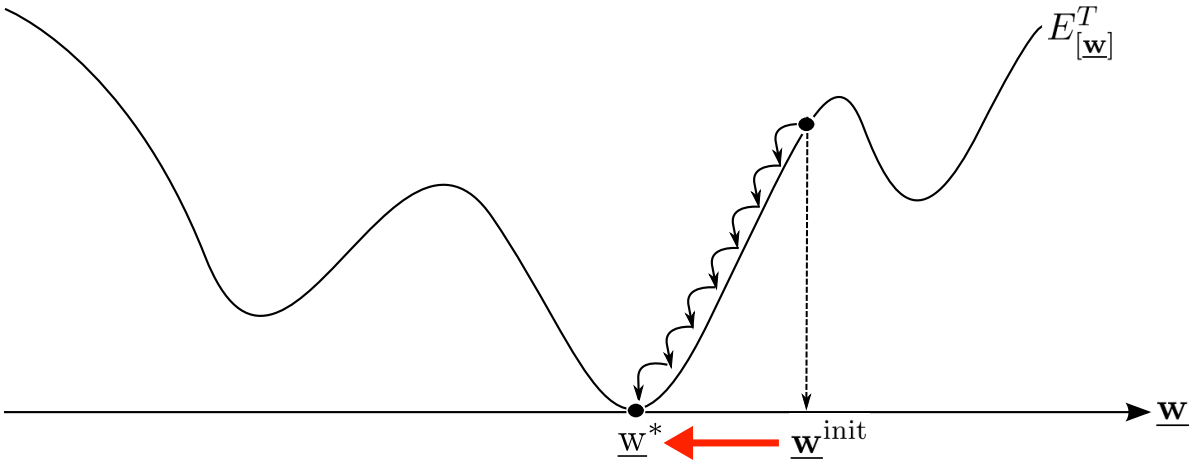
1. Compute the gradient w.r.t. $\underline{\mathbf{w}}$ by taking the first partial derivatives w.r.t to each component in the vector $\underline{\mathbf{w}}$

$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial \underline{\mathbf{w}}} = \left(\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_0}, \frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_1}, \ldots, \frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_N} \right)^\top$$

2. Set the gradient to zero: $\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial \underline{\mathbf{w}}} \stackrel{!}{=} \underline{\mathbf{0}}$
3. Solve for $\underline{\mathbf{w}}$ to find extrema.
4. Select solution corresponding to global minimum.

Caveat: Closed-form solution infeasible for complex models such as MLPs.
Instead: Iterative learning algorithm gradient descent.

Finding the minumum of $E^T_{[\underline{\mathbf{w}}]}$ iteratively

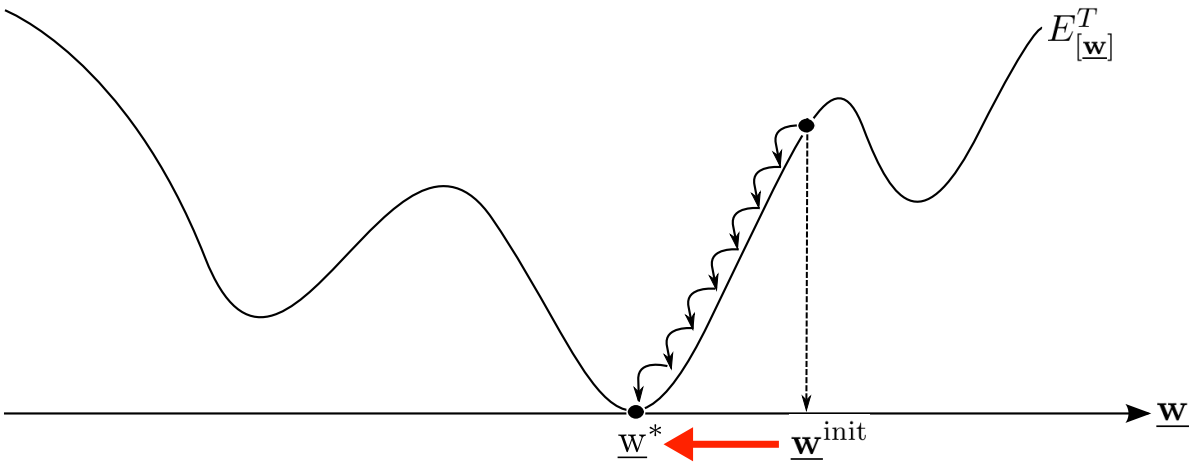


For our connectionist neuron example:

$$w_j(t+1) \quad = \quad w_j(t) \; - \; \underbrace{\eta}_{\substack{\text{learning}\\ \text{step}}} \cdot \underbrace{\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j}}_{\substack{\text{component of}\\ \text{gradient vector}}}$$

with $j = 0, \ldots, N$

Finding the minumum of $E^T_{[\underline{\mathbf{w}}]}$ iteratively



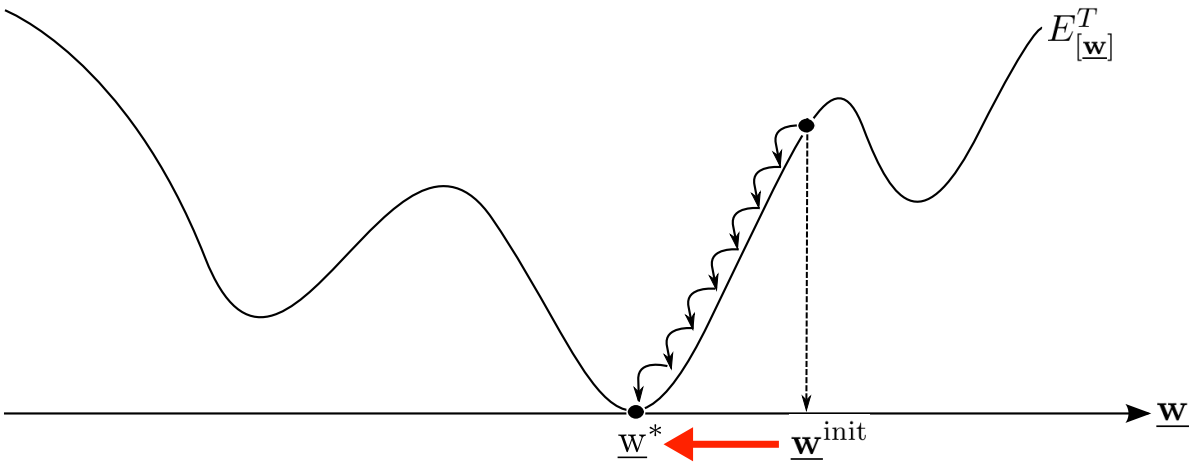
For our connectionist neuron example:

$$w_j(t+1) \quad = \quad w_j(t) \; - \; \underbrace{\eta}_{\substack{\text{learning}\\ \text{step}}} \cdot \underbrace{\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j}}_{\substack{\text{component of}\\ \text{gradient vector}}}$$

with $j = 0, \ldots, N$

Q: *Why do we <u>subtract</u> the gradient from $w_j(t)$?*

Finding the minumum of $E^T_{[\underline{\mathbf{w}}]}$ iteratively



For our connectionist neuron example:

$$w_j(t+1) \quad = \quad w_j(t) \; - \; \underbrace{\eta}_{\substack{\text{learning}\\ \text{step}}} \cdot \underbrace{\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j}}_{\substack{\text{component of}\\ \text{gradient vector}}}$$

with $j = 0, \ldots, N$

Q: *So what is the downside of using gradient descent?*

Gradient descent

So what's the downside of using gradient descent?

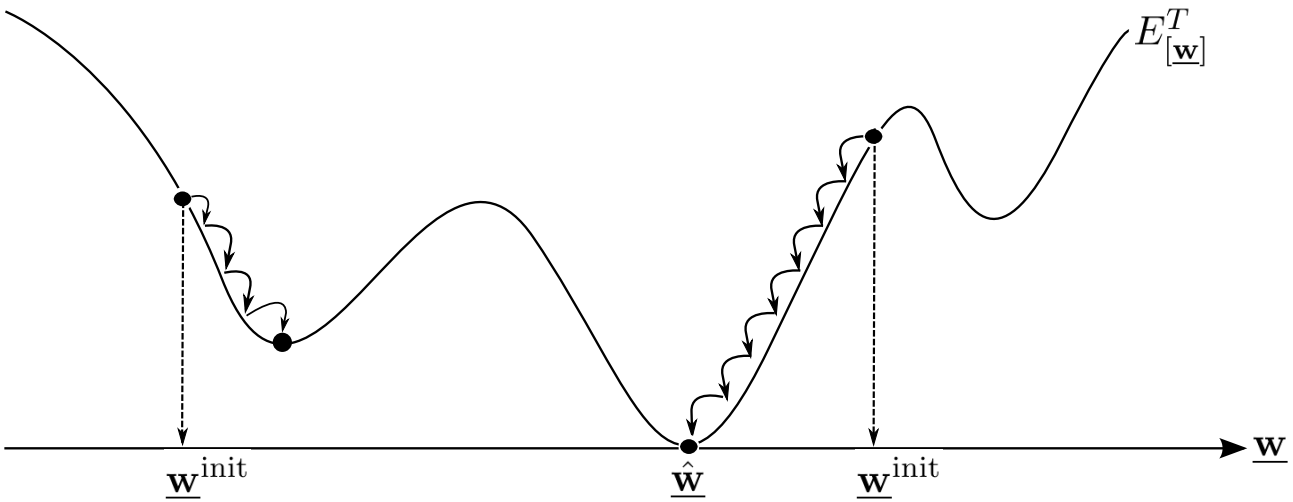
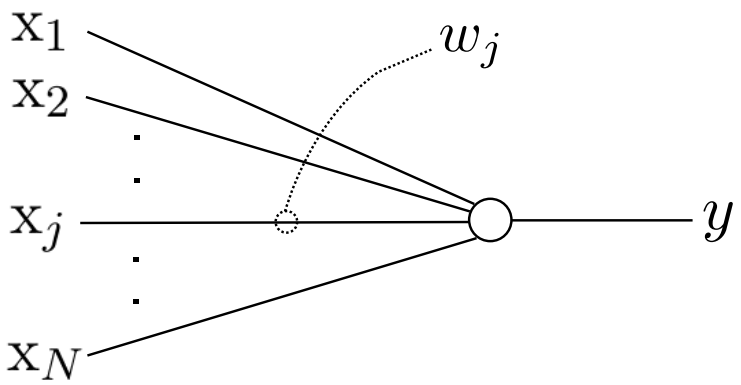


Figure: Gradient descent finds local minima. We therefore denote the solution with $\underline{\hat{\mathbf{w}}}$

Gradient calculation: Connectionist neuron



Working with the connectionist neuron, gradient descent updates the weights using:

$$w_j(t+1) \quad = \quad w_j(t) \; - \; \underbrace{\eta}_{\substack{\text{learning}\\ \text{step}}} \cdot \underbrace{\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j}}_{\substack{\text{component of}\\ \text{the gradient}}}$$

Knowing that

$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j} = \frac{1}{p}\sum_{\alpha=1}^{p} \frac{\partial e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}{\partial w_j} = \frac{1}{p}\sum_{\alpha=1}^{p} \frac{\partial e^{(\alpha)}}{\partial w_j}$$

with $j = 0, \ldots, N$.

$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j} = \frac{1}{p}\sum_{\alpha=1}^{p} \underbrace{\frac{\partial e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}{\partial y(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{w}})}}_{\substack{\text{factor depending}\\ \text{on cost function}}} \cdot \underbrace{\frac{\partial y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})}{\partial w_j}}_{\substack{\text{factor depending on}\\ \text{model class}\\ \text{(e.g. perceptron, MLP)}}}$$

quadratic error: $e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) := \frac{1}{2}\left(y_T - y(\underline{\mathbf{x}}; \underline{\mathbf{w}})\right)^2$

$$\frac{\partial\ e_{\text{quadr.}}\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}{\partial y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})} = -y_T^{(\alpha)} + y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})$$

with $j = 0, \ldots, N$.

$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j} = \frac{1}{p} \sum_{\alpha=1}^{p} \underbrace{\frac{\partial e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}{\partial y(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{w}})}}_{\substack{\text{factor depending} \\ \text{on cost function}}} \cdot \underbrace{\frac{\partial y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})}{\partial w_j}}_{\substack{\text{factor depending on} \\ \text{model class} \\ \text{(e.g. perceptron, MLP)}}}$$

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) := f\Big(\sum_{j=0}^{N} w_j x_j \Big) = f\left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}}\right) = f\left(h(\underline{\mathbf{x}}; \underline{\mathbf{w}})\right)$$

It follows:

$$\begin{aligned}
\frac{\partial y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})}{\partial w_j} &= \frac{\partial y(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{w}})}{\partial h(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})} \cdot \frac{\partial h(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})}{\partial w_j} \\
&= \underbrace{f'(h(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}))}_{\substack{\text{depends on} \\ \text{transfer function}}} \cdot \underbrace{\frac{\partial\ \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)}}{\partial w_j}}_{=x_j}
\end{aligned}$$

Again with less clutter:

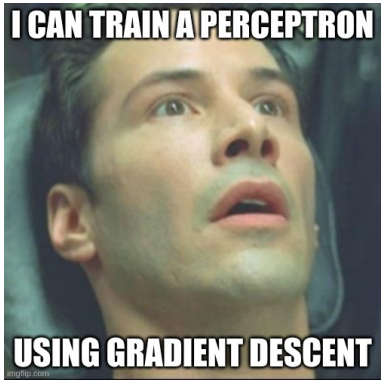
$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j} = \frac{1}{p}\sum_{\alpha=1}^{p} \underbrace{\frac{\partial e_{\text{quadr.}}\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}{\partial y(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{w}})}}_{\substack{\text{factor depending}\\ \text{on cost function}}} \cdot \underbrace{\frac{\partial y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})}{\partial w_j}}_{\substack{\text{factor depending on}\\ \text{model class}\\ \text{(e.g. perceptron, MLP)}}}$$

$$= \frac{1}{p}\sum_{\alpha=1}^{p} \left(y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}) - y_T^{(\alpha)}\right) \cdot f'\left(h(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})\right) \cdot x_j$$

Again with less clutter:

$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j} = \frac{1}{p}\sum_{\alpha=1}^{p} \underbrace{\frac{\partial e_{\text{quadr.}}\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}{\partial y(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{w}})}}_{\substack{\text{factor depending}\\ \text{on cost function}}} \cdot \underbrace{\frac{\partial y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})}{\partial w_j}}_{\substack{\text{factor depending on}\\ \text{model class}\\ \text{(e.g. perceptron, MLP)}}}$$

$$= \frac{1}{p}\sum_{\alpha=1}^{p} \left(y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}) - y_T^{(\alpha)}\right) \cdot f'(h(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})) \cdot x_j$$



Computing gradients for an MLP

Next time.